Message #1 (from S:>kmp>mail.kbin.newest)
Date: Mon, 26 Aug 1991 16:28 EDT
From: David A. Moon <moon@cambridge.apple.com>
To: kmp@Symbolics.com
Subject: Ralph Conditions (part 1 of 2)

Kent,

One thing I've done at Apple is to help out some people who are working with a new dialect of Lisp, currently called Ralph but soon to be renamed, by designing an exception handling facility for them. It's closely based on the Common Lisp condition system that you designed, but with some things rethought and some things simplified. In general, Ralph is a simpler and more compact language than Common Lisp, but a richer and more industrial language than Scheme. I thought you might like to see the design document for Ralph Conditions. If you have any comments on this, we'd be glad to listen to them, and might even change things based on your input if you point out something we've overlooked. On the other hand, if you don't have any comments that's okay too; you should have the document because it derives from your work, regardless of whether you want to give any input on it. We're only just starting to go public with this, so please don't show it to anyone else just yet. There will be a paper on Ralph by Andrew L.M. Shalit at the next POPL, assuming the program committee accepts the paper, and after that I'd like to publish a paper on the Ralph condition system, if I can find the time to write it up properly, so by January this will be extremely public but right now we don't want to attract a lot of attention. By the way the purpose of this language is not to replace Common Lisp; it's aimed at a rather different audience.
--Dave

Rest of this message would have been the design document if the mail system I'm using didn't blow dead goats. I'll have to mail it in 2 parts.

RALPH CONDITIONS (version 1.1)
David A. Moon
14 August 1991

Here's a proposal for conditions for Ralph, finally ready for distribution after countless iterations in Cambridge.

I'd like to write a paper explaining this in a way that's accessible to the Scheme community some time.

Revision history:

12-Aug-91	1.0	Moon	Initial release to Ralph-Designers
14-Aug-91	1.1	Moon	Fix restart-condition problemette, strategy #2 i.e. give <RESTART> the CONDITION: initarg, also fix poor English in overview of restarting

>>> BACKGROUND

A long-standing problem of software engineering is to develop an organized way to deal with exceptions, situations that must be handled gracefully but that are not conceptually part of the normal operation of the program. A number of programming languages contain linguistic features for exceptions; the ones I know about are Common Lisp, C++, PL/I, and Ada.

Of course it is possible to program exception handling without using any special linguistic features. For example, all functions could return an extra result that indicates whether they succeeded or failed, functions could take an extra argument that they consult if an exception occurs, or there could be a designated exception-handling function that gets called whenever a problem arises. All of these approaches have been used in one real-life system or another, but they are deficient in two ways. First, they are too informal and don't provide enough structure to allow an organized, systematic approach to exception handling. More importantly, the first two approaches do not provide any textual separation between "normal code" and "code for dealing with exceptions"; exception-related code is sprinkled throughout the program. This leads to two problems: one is the well-known mistake of forgetting to test error codes and thus failing to detect an exception, perhaps one that "could never happen." Second, program clarity is lost because it isn't easy to think about the main flow of the program while temporarily ignoring exceptions.

Thus the most important things about a linguistic exception handling facility are to provide overall structure, to eliminate the possibility of failing to notice an exception, and to provide a clean separation between "normal code" and "code for dealing with exceptions".

All exception systems involve the concept of "signal" (sometimes with a different name, such as "raise" or "throw") and the concept of "handle" (sometimes with a different name such as "on-unit" or "catch"). All exception systems considered here dynamically match signallers with handlers, first invoking the most recently established matching handler that is still active, and then if it declines to handle the exception invoking the next most recent matching handler, and so on. Thus exception systems are really a way of establishing a run-time connection between a signaller and a handler, as distinct from the usual fixed connection between a caller and a callee through function-name matching.

The major design choices in exception systems are name-based versus object-based, calling versus terminating, and formal versus ad-hoc recovery. These are discussed in the following paragraphs.

PL/I and Ada have name-based exception systems. One signals a name, and a handler matches if it handles the same name or "any". The name is a constant in the source text of the program, not the result of an expression.

C++ and Common Lisp have object-based exception systems. One signals an object, and a handler matches if it handles a type that that object belongs to. Object-based exceptions are more powerful because the object can communicate additional information from the signaller to the handler, because the object to be signalled can be chosen at run time rather than signalling a fixed name, and because type inheritance in the handler matching adds abstraction and provides an organizing framework.

The C++ object-based exception system has complicated inheritance rules and allows any object whatsoever to be used as an exception. Common Lisp, by contrast, allows only objects that inherit from a certain built-in class to be used as exceptions, thus limiting the inheritance rules to class inheritance and providing a class on which to hang any protocols specific to exceptions.

Ada and C++ have terminating exception systems: before a handler receives control, all dynamic state between the handler and the signaller is unwound, as if signalling was a non-local goto from the signaller to the handler. PL/I has a calling exception system: when a handler receives control, the signaller is still active and control can be returned to it, as if signalling was a function call from the signaller to the handler. Common Lisp has both, with the calling semantics regarded as fundamental and the terminating semantics explained in terms of it. Note that calling semantics need not imply that the signal operation can return the way a function call returns; Common Lisp provides a "restart" mechanism that signallers use to grant handlers permission to return. The Common Lisp model shows that terminating versus calling is a property of handlers, not of exception systems as a whole.

Terminating exception systems are okay for errors. However, they don't work for an exception that is not an error and doesn't require termination, either because there is a default way to handle it and recover or because it can safely be ignored by applications that don't care about it. Non-error exceptions are quite common in networked environments, in computers with gradually expiring resources such as batteries, in complex user interfaces, and as one approach for reflecting hardware exceptions such as page protection violations or floating point overflow to the application.

Most languages have not formalized how to recover from exceptions, leaving programmers to invent ad hoc mechanisms. Common Lisp provides "restarts", which are an object-based way of establishing a run-time connection from a handler back to the signaller or to any other part of the dynamically active program. A handler can choose to return control to any available restart, or else can mimic terminating semantics using the language's normal non-local-exit mechanisms. Restarts can also be a good way for programs to indicate to an interactive debugger what the end user's choices are for recovery from an exception that lands in the debugger. In this sense the debugger can be understood as just a complicated handler. Unfortunately some of the details of restarts are somewhat poorly designed in Common Lisp.

It is necessary to have a way to clean up when execution of a function is terminated by a non-local exit initiated either by the function itself or by something it explicitly or implicitly called. In C++ this includes invoking destructors for automatic objects, as well as application-specific cleanup. Ada, C++, and the Multics dialect of PL/I confuse this concept with exception handling. Common Lisp and Ralph realize it is related but different and call it unwind-protect.

Within Apple, exception facilities can also be found in MacApp, Leopard, and Tupperware. Pink uses the C++ language exception facility, currently in a kludge implementation that doesn't support all the features.

For detailed information on exceptions in Common Lisp, refer to chapter 29 of Common Lisp: the Language, Second Edition, by Guy L. Steele Jr. For detailed information on exceptions in C++ (not yet implemented in many C++ implementations, but adopted into the draft proposed ANSI C++ standard), refer to chapter 15 of The Annotated C++ Reference Manual, by Margaret A. Ellis and Bjarne Stroustrup. It should not be necessary to read either of these documents to understand this proposal, however they may be helpful as sources of background information.

>>> GOALS

A modest and simple proposal for conditions in Ralph.

I propose that Ralph should adopt an exception facility that is architecturally similar to the one in Common Lisp, but much simplified and reduced to a more modest size. Thus Ralph's exception facility will be object-based, use calling semantics but also provide terminating handlers, and provide formal recovery.

This should provide all the functionality of the Common Lisp condition system, but with less brain damage. The key differences are that a number of superfluous features have been removed, syntax is simpler and made consistent with Ralph, restarts and conditions have been unified into a single concept, and the relationship of a signaller to a handler is more like an ordinary function call.

This proposal is indeed simplified; I wrote a reference implementation in about 200 lines of code and comments.

>>> DEFINITIONS OF TERMS

condition -- an object that represents the occurrence of an exception

abstract class -- a class that is not intended to be instantiated directly, only its concrete subclasses should be instantiated

signal -- verb: an exception is reported by signalling a condition

handler -- a function that responds to the signalling of a condition

dynamic handler -- a handler that is established when control enters a certain form and remains active while control is inside that form

default handler -- a handler that is always active; dynamic handlers are tried before default handlers

serious exception -- an exception that requires intervention before normal program computation can resume

error -- a serious exception caused by something invalid about the program (as opposed to its environment); clarified below

warning -- a non-serious exception that is typically handled by displaying a message to the user and continuing.

terminate -- the concept of abandoning execution of a portion of the program by taking a non-local exit

recovery -- the concept of fixing the cause of an exception and then resuming execution of the program

restart -- a condition signalled by a handler to recover safely after

encountering an exception

restart handler -- a function that responds to the signalling of a restart
by recovering from the original exception

debugger -- the handler of last resort, whether it's an interactive debugger,
a "core dumper", a "bomb icon", or something else

>>> OVERVIEW

Conditions are a dynamically-bound, type-based mechanism for finding and invoking functions. When an exceptional situation occurs, it is indicated to the caller by signalling a condition. The condition facility locates an applicable handler and calls it. An applicable handler is one that matches the signalled condition by type and by an (optional) test function associated with the handler. The condition system is simply a way for a signaller and a handler to be introduced to each other. Once they have met, they communicate through an ordinary function call.

Like any function, the called handler either returns some values or takes a non-local exit. Either way, the handler has handled the condition and the act of signalling is completed. A dynamic handler also has the option of declining to handle the condition, passing the buck to the next applicable handler, by calling a next-handler function passed to the dynamic handler as an argument. A default handler cannot decline since it is always the last applicable handler. Every signalled condition is handled, because the system ensures that there is always an applicable default handler.

For the next few paragraphs we need to define terms for portions of the dynamic state of execution. The (rather poor) names I chose come from the stack model of function calling.

outside stack -- the state existing just before the handler was established

signalling unit -- the module or other conceptual program component that includes the form that signalled the condition and does not include the form that established the handler. This informal concept gives us a notion of where the interface boundary between the signaller and the handler lies.

middle stack -- the state existing just before the signalling unit was called, minus the outside stack

inside stack -- the state existing just before signalling occurred, minus the middle stack

The simplest way to handle a condition is to terminate the signalling unit by taking a non-local exit established in the outside stack. The convenient HANDLER-CASE macro is provided for this purpose.

A less common handling style is to terminate the signalling unit by taking a non-local exit established in the middle stack, leaving the handler in force.

Instead of terminating, a handler can recover, returning control to the signalling unit. This can be done either by returning values that the signalling unit will understand or by taking a non-local exit established in the inside stack. Returning or exiting can be done directly, or a more formal recovery mechanism called restarting can be used. The advantage of the formal mechanism is the advantage of any formal interface: it provides more assurance that the handler and the signalling unit agree on the meaning of what they are doing, and it provides some isolation of the handler from names and data representations internal to the signalling unit.

Returning values or non-locally exiting directly, rather than restarting, is a less formal mechanism with an increased likelihood of accidentally violating the protocol or contract between a signaller and a handler. Nonetheless it is allowed, in the spirit of freedom and because it is the primitive upon which restarting is built. When the direct mechanism is used, it should be used with due care.

A handler restarts by signalling a restart. Any values needed for recovery are passed in the restart (that is, in initialization arguments that the restart remembers either in slots or in any other way it chooses). The restart is handled by a restart handler which either returns or takes a non-local exit. If the restart handler returns some values, SIGNAL returns those values and

the handler that called SIGNAL also returns them. The call to SIGNAL from the signalling unit that signalled the original condition returns the same values, and the signalling unit recovers as directed by those values. If the restart handler was established by the signalling unit, then it was established in the inside stack and typically takes a non-local exit in the inside stack. It is just as valid to invoke a restart handler established in the outside or middle stack by a program component other than the signalling unit.

For every condition class, there should be a "recovery protocol" which says what is the meaning of handling by returning, what the values returned mean, and what restart handlers are supposed to be established by the signalling unit. The recovery protocol tells the handler what it can expect from the signaller. For many condition classes, this is the empty protocol: handling by returning isn't allowed and no particular restart handlers are provided. In this case only handling by terminating is possible. (Terminating might be accomplished by signalling a restart whose handler was established in the outside or middle stack, or by an ordinary non-local exit.) The recovery protocol for a subclass should be compatible with the recovery protocol of a superclass; I will omit a formal definition of protocol compatibility for now.

An example recovery protocol for a hypothetical <UNBOUND-SLOT> condition could be: returning a value uses that value as if it had been the contents of the slot; a restart handler for <NEW-VALUE> is available; <NEW-VALUE> has initialization arguments VALUE:, the value to use, and PERMANENT:, #t to store the value into the slot or #f (the default) to leave the slot unbound.

At present no formal mechanism is provided for describing recovery protocols; they are left to the documentation of a condition class. Perhaps such a mechanism might be added in the future when protocols in general are better understood. Introspective functions are provided for discovering what recovery facilities are actually available, but this is different from (one hopes a superset of) the recovery facilities that are guaranteed by a recovery protocol to be available.

The debugger is the condition handler of last resort that receives control if no program-provided handler handles a serious condition. (This is true even if the "debugger" provided cannot analyze or intervene in the execution of programs, but can only abort or restart them. The debugger might be merely a "core dumper", a "bomb icon", or something similar.) An interactive debugger ought to offer the user the ability to signal any restart for which a restart handler is applicable and to return if the condition's recovery protocol allows it; this could be done with a menu titled "Recovery."

>>> NAMING APOLOGY

Several names herein were chosen for no better reason than that they are the names that Common Lisp used for similar concepts, operators, or classes. That's not such a bad reason; at least by following a standard there is some chance of Lisp programmers already being familiar with the names, and in addition we know that significant thought went into the choice of names. If we can come up with names that are so clearly superior that they outweigh Common Lisp's armamentarium of standards, textbooks, and courses, I have no objections at all to changing the names.

>>> SPECIFICATION: MAIN PART

>>> Classes

A condition is a general instance of the abstract class <CONDITION>.
It is an object that represents the occurrence of an exception.
Conditions have indefinite extent and no special restrictions.
A program that does not encounter any exceptional situations will
not implicitly allocate any conditions.

There is a default handler for <CONDITION> that returns #f.

<SERIOUS-CONDITION> is the abstract class of all conditions that cannot safely be ignored. There is a default handler for <SERIOUS-CONDITION> that calls the debugger, using an unspecified mechanism.

<ERROR> is the abstract class of all conditions that represent something invalid about the program (as opposed to environmental conditions such as running out of memory or battery power, or inability to establish a network connection, for example). <ERROR> is a subclass of <SERIOUS-CONDITION>.

<ERROR> is distinct from <SERIOUS-CONDITION> so one can establish a handler for errors that not also trap unpredictable environmental exceptions such as network problems.

<SIMPLE-ERROR> is the concrete class of error conditions that consist of just an error message constructed from a FORMAT string and arguments; it is a subclass of <ERROR>. The recovery protocol is empty. The initialization arguments FORMAT-STRING: and FORMAT-ARGUMENTS: are accepted. The accessor functions CONDITION-FORMAT-STRING and CONDITION-FORMAT-ARGUMENTS are applicable.

<TYPE-ERROR> is a concrete subclass of <ERROR> used by CHECK-TYPE. The recovery protocol is empty. The initialization arguments VALUE: and TYPE: are accepted. The accessor functions TYPE-ERROR-VALUE and TYPE-ERROR-EXPECTED-TYPE are applicable.

<WARNING> is an abstract subclass of <CONDITION>. There is a default handler for <WARNING> that displays the warning in a user-interface dependent way and then returns #f. The recovery protocol is that any value can be returned and will be ignored.

<SIMPLE-WARNING> is a concrete subclass of <WARNING> used by SIGNAL. The recovery protocol is the same as for <WARNING>. The initialization arguments and accessor functions are the same as for <SIMPLE-ERROR>.

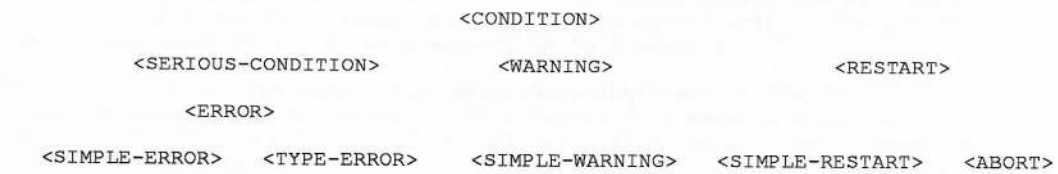
A restart is a general instance of the abstract class <RESTART>. <RESTART> is a subclass of <CONDITION>. There is a default handler for <RESTART> that signals an error reporting an attempt to use a restart for which no restart handler was established. The recovery protocol concept is not applicable to restarts. The initialization argument CONDITION: is accepted and ignored by <RESTART>; some subclasses save the value of this initialization argument, and use it to associate a restart with a particular condition from which the restart can recover. At present none of these subclasses are defined as part of the language. Other restarts don't care, they can recover from any condition.

<SIMPLE-RESTART> is a concrete subclass of <RESTART> that exists just to be distinct from the other subclasses of <RESTART>; it is used by CERROR.

<ABORT> is a concrete subclass of <RESTART> whose handlers are expected to terminate execution of the current application command, or similar unit of execution, and return control to something like an application command loop. This is comparable to command-period on the Macintosh. The exact details of this depend on the particular environment, of course, but signalling an instance of <ABORT> is a uniform way to "get out."

Numerous condition classes specific to particular Ralph built-in functions or user functions will no doubt exist. Note that there can be other direct subclasses of <CONDITION> besides the three specified ones.

A diagram of the condition classes will go here and will look something like this, with circles and arrows added:



>>> Basic operator for signalling

SIGNAL condition [Sealed Function]

Signal the condition, trying each active dynamic handler, most recent first. If all dynamic handlers decline, calls (DEFAULT-HANDLER condition). If a handler returns, all the values that it returned are returned from SIGNAL. If SIGNAL returns when condition's recovery protocol does not allow returning, some handler has violated protocol; SIGNAL does not check for this error. If condition is a restart, the caller of SIGNAL should always assume that it might return.

See note below about the "dynamically active handler" concept.

Additional operators for signalling are defined for convenience. They are described in a later section.

>>> Basic operator for handling conditions

HANDLER-BIND (type function #key test description) body... [Macro]

Establishes a condition handler during the dynamic extent of the evaluation of the body forms, and returns the values of the last body form, or #f if the body is empty. Type, function, test, and description are evaluated before evaluation of the body begins.

Type - this handler applies to conditions that are general instances of this type. Normally this argument is a class, however if Ralph defines types that are more general than just classes, they will work here.

Function - a function that is called to handle a condition that matches type and passes test. The function receives two arguments typically named condition and next-handler. The function handles the condition by taking a non-local exit, returning values according to the condition's recovery protocol, or tail-recursively calling SIGNAL of a restart. The function can decline to handle the condition by tail-recursively calling next-handler with no arguments. The two calling possibilities are described as tail-recursive to ensure that all values returned by the call are returned by the handler. Not returning all the values could interfere with the condition's recovery protocol. A handler that -really- knows what it is doing could use a non-tail-recursive call, however anything that knows what it's doing in this situation is "obviously" unmodular. Note that a handler might not know the full recovery protocol because the condition's class might be a subclass of the handler's expected type.

Test - a function that is called with a condition that matches type, and returns true if this handler applies to this condition. The default is a function that always returns true. An example usage is a restart handler that is only for restarting from a particular condition, for example restarting from a hypothetical unbound-slot error by setting the slot and retrying the invocation of the accessor. The <set-and-continue> restart condition will have the signalled <unbound-slot> condition in a slot, and the handler's test will check for it. (Those class names are invented for this example and are not part of the specification.)

Description -- a function that is called with one argument, a stream, and outputs a brief description of this handler to the stream. This can be textual, in which case it should be one line with no newline or delimiters, or it can be graphical (anything that the stream allows). Descriptions are probably used only in restart handlers. The description argument can also be a string, which is an abbreviation for a function that writes that string to the stream. If description is omitted, some sort of default description will be concocted.

Test and function are distinct so that handler applicability can be tested without actually handling (which might take a non-local exit). One use for this is constructing a list of available restart handlers.

There is no "condition wall", i.e. when executing function the set of available handlers is not reset to the handlers that were in effect when the HANDLER-BIND was entered. This difference from Common Lisp is necessary to make restarts work.

The expansion of a HANDLER-BIND form is implementation-dependent. Implementations are encouraged to use an expansion that optimizes establishing a handler for both speed and space, even if that increases the cost of signalling. The assumption is that most of the time a handler will never be used because the exception it is looking for will never occur.